

Supervised Learning

Supervised learning itu belajar dari data yang udah ada "jawaban"-nya (label). Model dilatih pakai pasangan input-output, terus dipakai buat nebak output dari data baru yang belum pernah dilihat. Ada dua jenis tugas utama: klasifikasi (nebak kategori, misal spam/bukan spam) dan regresi (nebak angka, misal harga rumah).

1. Generalization, Overfitting, Underfitting

Tujuan utama model bukan cuma jago di data latihan, tapi juga harus jago di data baru, ini yang disebut generalization.

- **Overfitting** : Model kebanyakan hafal detail/noise data latihan sampai jelek performanya di data baru.
- **Underfitting** : Model kelewat sederhana, sampai pola penting di data pun nggak ketangkap.

Makin kompleks model, makin gampang overfitting, apalagi kalau datanya sedikit. Jadi kompleksitas model harus disesuaikan sama jumlah dan variasi data yang ada.

2. k-Nearest Neighbors (k-NN)

Algoritma paling sederhana: buat nebak data baru, model lihat k tetangga terdekat di data latihan, terus ambil suara terbanyak (klasifikasi) atau rata-rata (regresi).

- **k kecil** : k kecil (misal $k=1$) → model sangat fleksibel tapi gampang overfitting.
- **k besar** : k besar → keputusan makin halus/general, tapi bisa underfitting kalau kebesaran.
- **Catatan** : Gampang dipahami dan sering jadi baseline yang oke, tapi lambat kalau datanya besar dan kurang bagus buat data dengan banyak fitur (fitur jarang/sparse).

3. Linear Models

Model linear bikin prediksi dari kombinasi linear fitur-fiturnya (bobot dikali fitur, dijumlahin).

Untuk Regresi

- **Linear Regression (OLS)** : Regresi paling dasar, cari garis/bidang yang paling pas ke data. Gampang overfitting kalau fiturnya banyak.
- **Ridge** : Mirip linear regression tapi bobotnya "ditekan" biar nggak terlalu besar (regularisasi L2), bikin model lebih general.
- **Lasso** : Mirip Ridge, tapi bisa bikin sebagian bobot fitur jadi persis nol (regularisasi L1), otomatis milih fitur yang penting aja.

Untuk Klasifikasi

- **Logistic Regression & Linear SVM** : Bikin garis/batas pemisah antar kelas. Dua model paling umum, sama-sama pakai regularisasi biar nggak overfitting.

- **Multiclass** : Buat lebih dari 2 kelas, biasanya dipecah jadi beberapa model "satu kelas vs sisanya" (one-vs-rest).

Model linear cepat dilatih dan diprediksi, cocok buat data besar atau berdimensi tinggi, tapi kurang jago kalau hubungan fitur dan targetnya nggak lurus (non-linear).

4. Naive Bayes

Mirip model linear tapi lebih simpel dan lebih cepat dilatih, masing-masing fitur dihitung kontribusinya sendiri-sendiri (diasumsikan saling bebas). Cocok banget buat data teks berdimensi sangat tinggi, walau akurasinya biasanya sedikit di bawah logistic regression.

5. Decision Tree

Model bikin serangkaian pertanyaan ya/tidak tentang fitur (misal "umur > 30?") buat memecah data sampai tiap bagian jadi murni satu kelas/nilai.

- **Kelebihan**: Gampang divisualisasikan dan dijelaskan ke orang awam, nggak butuh scaling fitur.
- **Kelemahan** : Kalau dibiarkan tumbuh terus, gampang banget overfitting, sehingga perlu dibatasi kedalamannya (pruning).

Decision tree juga bisa kasih tau fitur mana yang paling penting lewat `feature_importances`.

6. Ensemble: Random Forest & Gradient Boosting

- **Random Forest** : Gabungan banyak decision tree yang masing-masing dilatih agak beda-beda (random), hasilnya dirata-ratakan. Lebih tahan overfitting dibanding satu decision tree doang.
- **Gradient Boosting** : Bikin tree satu per satu secara berurutan, tiap tree baru belajar memperbaiki kesalahan tree sebelumnya. Biasanya lebih akurat dari Random Forest tapi lebih sensitif ke setting parameter dan butuh waktu latihan lebih lama.

Keduanya termasuk model paling sering dipakai di kompetisi/industri karena akurasinya tinggi tanpa perlu banyak preprocessing data.

7. Kernelized SVM

Pengembangan dari Linear SVM buat menangani data yang batas antar kelasnya nggak berupa garis lurus. Triknya (kernel trick) "memproyeksikan" data ke dimensi lebih tinggi secara matematis tanpa perlu menghitungnya secara eksplisit, biar jadi bisa dipisah garis lurus di sana.

Perlu di-scaling dulu fiturnya (semua fitur harus dalam rentang yang mirip), dan agak susah di-tuning (parameter C dan gamma), tapi bisa sangat akurat buat data yang jumlah fiturnya nggak terlalu banyak.

8. Neural Network (Multilayer Perceptron)

Terdiri dari beberapa "lapisan" unit yang masing-masing menghitung kombinasi linear dari input, dilewatkan fungsi non-linear, lalu diteruskan ke lapisan berikutnya. Makin banyak lapisan dan unit,

model makin kompleks dan bisa menangkap pola yang lebih rumit, tapi butuh data yang cukup banyak dan tuning yang lebih rumit dibanding model lain (jumlah lapisan, jumlah unit, regularisasi alpha, dll).

9. Estimasi Ketidakpastian dari Classifier

Selain sekadar nebak kelas, banyak model bisa juga ngasih tau seberapa "yakin" dia sama prediksinya.

- **decision_function()** : Angka yang nunjukin seberapa yakin model condong ke satu kelas, makin besar/negatif nilainya, makin yakin model.
- **predict_proba()** : Ngasih probabilitas tiap kelas (jumlahnya selalu 1), jadi lebih gampang diartikan dibanding decision_function.

Kesimpulan

Algoritma	Cara Kerja Singkat	Kelebihan / Catatan
k-NN	Lihat tetangga terdekat	Simpel, cocok data kecil
Linear Model	Kombinasi linear fitur	Cepat, data besar/dimensi tinggi
Naive Bayes	Fitur dihitung independen	Sangat cepat, cocok data teks
Decision Tree	Serangkaian pertanyaan ya/tidak	Mudah dijelaskan, gampang overfit
Random Forest	Banyak tree digabung (rata-rata)	Akurat, tahan overfitting
Gradient Boosting	Tree dibangun bertahap perbaiki error	Sangat akurat, sensitif parameter
Kernel SVM	Proyeksi ke dimensi lebih tinggi	Kuat untuk batas non-linear
Neural Network	Banyak lapisan unit non-linear	Kompleks, butuh data & tuning besar